

Helix Thready — Web Portal Guide

Table of contents

1. Accessing the portal

2. Information architecture (diagram)

3. Login & MFA

4. Threads & posts

5. Semantic search

6. Administration screens

7. Real-time updates

8. Accessibility, SEO & performance bar

9. Desktop (Tauri) notes

10. Tutorials

11. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (user-guides)	Initial Angular portal guide
2	2026-07-22	swarm (user-guides, Pass 3)	Depth pass: split the information-architecture diagram explanation into multi-paragraph form; added a screen-by-screen reference table and linked quickstart.md

The Web portal is the **primary management surface** (`[OPERATOR]` Web+CLI first). It is an **Angular** app on the shared **OpenDesign** `design_system` (Angular 19 for the product client; Angular 22 for marketing/SSG) `[IN-HOUSE]` `[CONSTITUTION §11.4.162]`. Everything a user or admin needs — browsing threads, semantic search, assets, administration — is here. It consumes the same REST `/v1` + event bus as the CLI/TUI.

VERIFIED vs ASSUMPTION. Angular + OpenDesign is VERIFIED from the decision matrix. Exact screen names/routes below are this guide’s proposal `[DEFAULT — adjustable]`, aligned to the **design area** once its wireframes are published. `design_system` is FOUNDATION (web-only, extracted from HelixOTA) `[GAP register §8.1]` — the Thready brand theme layer is a `P1`.

Table of contents

1. Accessing the portal

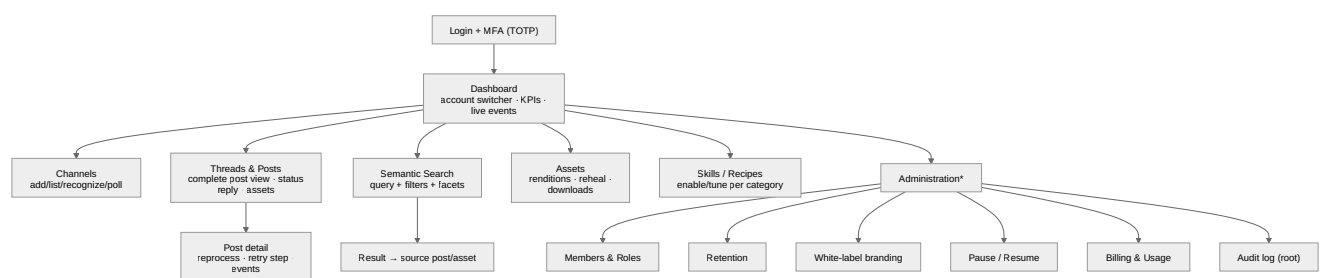
2. Information architecture (diagram)
3. Login & MFA
4. Threads & posts
5. Semantic search
6. Administration screens
7. Real-time updates
8. Accessibility, SEO & performance bar
9. Desktop (Tauri) notes
10. Tutorials
11. Open items

1. Accessing the portal

Environment	URL
Production	https://thready.hxd3v.com
Staging	https://sta.thready.hxd3v.com
Development	https://dev.thready.hxd3v.com

Each subdomain routes to a fully isolated stack (final request §8.2). Use a modern evergreen browser; HTTP/3 is used when available with HTTP/2 fallback.

2. Information architecture (diagram)



Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: [diagrams/web-portal-ia.mmd](#).

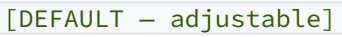
Explanation (for readers/models that cannot see the diagram). After login and TOTP MFA the user lands on the **Dashboard**, which is the hub of the whole IA. It carries an account switcher (for people who belong to multiple Accounts), headline KPIs, and a live event feed. The account switcher is load-bearing: because a person can hold different roles in different Accounts, the switcher is what scopes every subsequent screen to a single tenant.

From the dashboard the primary navigation fans out to the consumption surfaces. **Channels** lets you add/list channels and see auto-recognition and poll settings. **Threads & Posts** is the complete-post reading experience, with each post's status reply and linked assets. **Semantic Search** is a query box with structured filters and facets. **Assets** handles renditions, reheal, and downloads. And **Skills/Recipes** lets an Admin enable and tune the per-hashtag recipes. These five are what a Standard User spends nearly all their time in.

An **Administration** section marked  is visible only to Admin tiers. It expands to Members & Roles, Retention, White-label branding, processing Pause/Resume, Billing & Usage, and — for the Root Admin only — the Audit log. Visibility here is RBAC-driven: the section does not merely disable controls, it does not render for users who lack the role, and the server rejects the underlying calls regardless of what the UI shows.

Two drill-down paths are shown explicitly because they are the ones users follow most. A thread opens a **Post detail** view with reprocess/retry-step controls and that post's event history. A search result links straight back to its **source post or asset** — so search is a navigation *entry point*, not a dead-end results list. This “search deep-links to source” behaviour is what makes semantic search feel like part of the app rather than a bolt-on.

The IA mirrors the **CLI command tree** and the **TUI views** one-to-one, which is why a workflow learned in the portal transfers directly to the terminal surfaces and vice-versa.

Screen-by-screen reference.  routes align to the **design area** wireframes on publish ().

Screen	Route (proposed)	Min role	Primary actions	CLI equivalent
Dashboard	<code>/</code>	user	switch account, view KPIs, watch live feed	<code>thready events tail</code>
Channels	<code>/channels</code>	account_admin	add/list/recognize/poll	<code>thready channel ...</code>
Threads & Posts	<code>/threads</code>	user	browse complete posts, open detail	<code>thready thread ...</code>
Post detail	<code>/threads/:id</code>	user	reprocess, retry step, view events	<code>thready post ...</code>
Search	<code>/search</code>	user	query + filters, deep-link to source	<code>thready search</code>
Assets	<code>/assets</code>	user	list renditions, reheal, download	<code>thready asset ...</code>
Skills / Recipes	<code>/skills</code>	account_admin	enable/disable/tune per hashtag	<code>thready skill ...</code>
Members & Roles	<code>/admin/members</code>	account_admin	invite/remove/set role	<code>thready member ...</code>
Retention	<code>/admin/retention</code>	account_admin (global: root)	set per-account default	<code>thready retention ...</code>
Branding	<code>/admin/branding</code>	account_admin (if enabled)	color/logo/slogan	<code>thready brand set</code>
Processing	<code>/admin/processing</code>	account_admin (global: root)	pause/resume	<code>thready processing ...</code>
Billing & Usage	<code>/admin/billing</code>	account_admin (all: root)	view meter	<code>thready billing ...</code>
Audit log	<code>/admin/audit</code>	root	query/export append-only log	<code>thready audit ...</code>

3. Login & MFA

1. Enter email + password (Argon2id-hashed server-side, ≥12 chars, breach-checked).
2. Enter your TOTP code — **mandatory for Admin tiers**, optional for Standard Users (`THREADY_MFA_REQUIRED_TIERS`).
3. Sessions: access token 15 min, refresh 7 d, idle logout 30 min (Aggressive posture, Q9).

First-time Admins are forced through TOTP enrolment before any other screen loads.

4. Threads & posts

The Threads view lists complete posts per channel. Opening a post shows:

- The **complete post**: root message + full organic reply chain (system replies excluded).
- Detected **categories** (hashtags + content type), including any inferred by indirect determination.
- The **status reply** Thready posted back (success/metrics/asset refs).
- **Linked assets** (raw + `-web` renditions) resolved via the Asset Service, and any generated **research** documents.
- **Processing state** with `Reprocess` and `Retry step` actions (client → REST → System).

5. Semantic search

A single query box backed by pgvector + a real llama.cpp embedder searches **both** original posts and generated materials by meaning (< 500 ms target). Filters: kind (post/asset/research), account, date range, category, sensitivity. Each result deep-links to its source.

[GAP: 1] If the deployment is running on HelixLLM’s default **hash embedder**, search relevance is garbage. The portal surfaces an amber banner “*Semantic search degraded: non-semantic embedder*” and links to [troubleshooting.md §5](#).

6. Administration screens

Visible per RBAC:

- **Members & Roles** — invite/remove, set `account_admin` / `user` (Account scope); Root can edit any.
- **Retention** — per-account default (shorten within Root’s cap); Root sets global.
- **White-label branding** — colors/logo/slogan per Account; Helix attribution persists in footers.
- **Processing** — pause/resume (Account scope for Admins, global for Root).
- **Billing & Usage** — the metering view (own account for Admins; all for Root).
- **Audit log** — append-only, queryable (Root Admin).

These map to the [Account Admin](#) and [Root Admin](#) guides.

7. Real-time updates

The portal subscribes to the event bus over WebSocket (SSE fallback). Lists and the post detail view update live as `post.received` / `post.processed` / error events arrive; sticky events (last-value with invalidation) rehydrate state on load, and durable replay covers reconnects (final request §3.4). The complete set of topics the portal consumes — and which are one-time vs sticky — is the consolidated [event catalog](#).

```
// Illustrative Angular service (final contract in ../api/index.md)
this.events.subscribe('post.processed', (e: PostProcessed) => {
  this.store.upsertPost(e.postId, { status: 'processed', assets: e.assets });
});
```

8. Accessibility, SEO & performance bar

Websites/portal meet the engineering-quality bar `[CONSTITUTION §11.4.190]`: fully responsive, **WCAG AA**, Core Web Vitals, page < 1.5 s (Aggressive SLO), semantic HTML, and for marketing surfaces SEO-complete (meta/OG/Twitter, canonical, schema.org/JSON-LD, robots+sitemap). Proven with captured screenshots + Lighthouse and visual-regression tests (`VisualRegression` LLM-vision + `ScreenDiff` pixel). Light + dark themes mandatory with explicit choice + system default.

Localization. The portal UI is fully localized in **English, Russian, and Serbian Cyrillic** (`en / ru / sr-Cyrl`) via HelixTranslate + `digital.vasic.i18n` (Q35). A language switcher is available in the account/profile menu; the default is `THREADY_DEFAULT_LOCALE` (`configuration.md §15`). Post content stays in its original language with on-demand translation; only the UI chrome is translated.

9. Desktop (Tauri) notes

The Desktop app is a **Tauri 2** shell (Rust core + the same Angular UI) — the org standard `[DEFAULT – adjustable]` (Q18). It adds OS-specific features (native notifications for `post.processed`, file-system asset export, tray controls for pause/resume) but is otherwise the portal. Install per OS from releases (`installation.md §8`).

10. Tutorials

Tutorial A — First login as an Account Admin. 1. Open your account URL → sign in → enrol TOTP. 2. Dashboard → **Channels** → *Add channel* → paste a Telegram invite link → save. 3. Wait for auto-recognition; watch the live feed for `post.received`. 4. Dashboard → **Members** → invite your team as `user`.

Tutorial B — Investigate and re-run a failed post. 1. **Threads** → filter status `failed` → open the post. 2. Read the status reply's error; click **Retry step** → choose the failed step, or **Reprocess** for a full refresh. 3. Confirm via the live event feed that it reaches `post.processed`.

Tutorial C — Brand your account. 1. **Administration** → **White-label branding** (if enabled by Root). 2. Set primary color, upload light+dark logos, set slogan → save. 3. Reload — header, generated-doc headers/footers, and status replies pick up the brand; the Helix Development footer attribution remains.

11. Open items

- [OPEN: web-1] Screen names/routes are [DEFAULT – adjustable]; align with the design area wireframes once published. Tracked: **ATM — reconcile portal IA with Figma.**
 - [OPEN: web-2] design_system is web-only FOUNDATION; the Thready brand theme + npm publish are P1 [GAP register §8.1]. Tracked: **ATM — Thready theme on design_system.**
 - [OPEN: web-3] Angular product version (19 vs 22) for Thready's product client is a [DEFAULT – adjustable] recommendation (Q17); confirm before build.
-

Made with love ♥ by Helix Development.